Rapport de Travaux Pratiques

Architectures Logicielles

Module :	Architectures Logicielles
Encadrant :	Pr. BE ELBAGHAZAOUI
Établissement :	ENSA Beni Mellal
Année :	2025 – 2026
Étudiant(e) :	ILYAS MACHHOUL
N° Apogée :	23012747

Chapitre 2

TP2 — Monolithe Modulaire

2.1 Objectifs

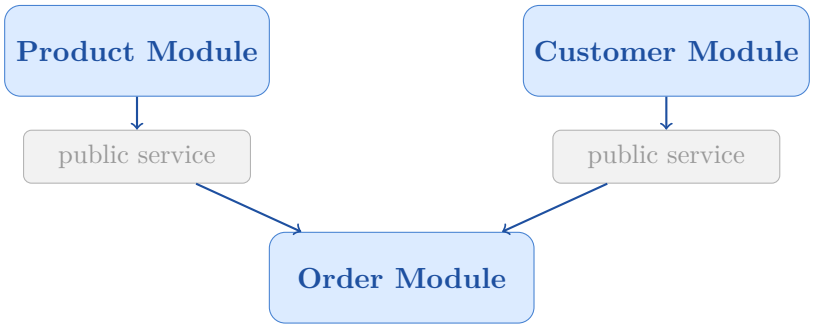
Objectifs du TP

- Restructurer le monolithe en modules métier distincts (Product, Customer, Order).
- Définir des *bounded contexts* et des interfaces publiques entre modules.
- Appliquer les design patterns : **DTO**, **Mapper** (MapStruct), **Service Layer**.
- Découpler les couches en séparant les entités internes des objets de transfert.
- Préparer une architecture propice à une future migration vers les microservices.

2.2 Architecture du Système

Architecture

Le monolithe modulaire organise le code en **domaines métier autonomes**. Chaque module expose uniquement ses services publics.



Règle de dépendance : Order Module consomme les *services publics* de Product et Customer — jamais leurs repositories directement.

2.2.1 Structure du projet

```
com.ecommerce.monolith
          product
                controller   ProductController.java
                service      ProductService.java / ProductServiceImpl.java
                repository   ProductRepository.java
                model        Product.java
                dto          ProductDTO.java / ProductRequest.java
                mapper       ProductMapper.java
          customer
                controller   CustomerController.java
                service      CustomerService.java / CustomerServiceImpl.java
                repository   CustomerRepository.java
                model        Customer.java
                dto          CustomerDTO.java / CustomerRequest.java
                mapper       CustomerMapper.java
          order
              controller   OrderController.java
              service      OrderService.java / OrderServiceImpl.java
              repository   OrderRepository.java
              model        Order.java
              dto          OrderDTO.java / OrderRequest.java
              mapper       OrderMapper.java
```

Listing 2.1 – Structure modulaire du projet

2.3 Code Source Important

2.3.1 DTO — ProductDTO et CreateProductRequest

```java
@Data
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private Integer stock;
}
```

Listing 2.2 – ProductDTO.java — Objet de transfert

```java
@Data
public class CreateProductRequest {

```

```
    @NotBlank(message = "Product name is required")
    private String name;

    private String description;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    private BigDecimal price;

    @PositiveOrZero(message = "Stock cannot be negative")
    private Integer stock = 0;
}
```

Listing 2.3 – CreateProductRequest.java — Requête de création

2.3.2 Mapper MapStruct

```
@Mapper(componentModel = "spring")
public interface ProductMapper {

    // Convertit une entite Product en DTO
    ProductDTO toDTO(Product product);

    // Convertit une liste d'entites en liste de DTO
    List<ProductDTO> toDTOList(List<Product> products);

    // Convertit une requete en entite
    Product toEntity(CreateProductRequest request);

    // Met a jour une entite existante
    @Mapping(target = "id", ignore = true)
    void updateEntity(CreateProductRequest request,
                      @MappingTarget Product product);
}
```

Listing 2.4 – ProductMapper.java — Interface MapStruct

2.3.3 Interface Service

```
public interface ProductService {
    List<ProductDTO>  getAllProducts();
    ProductDTO        getProductById(Long id);
    ProductDTO        createProduct(CreateProductRequest
       request);
    ProductDTO        updateProduct(Long id,
       CreateProductRequest req);
    void              deleteProduct(Long id);
}
```

Listing 2.5 – ProductService.java — Interface publique du module

2.3.4 Implémentation du Service

```java
@Service
@RequiredArgsConstructor
public class ProductServiceImpl implements ProductService {

    private final ProductRepository repository;
    private final ProductMapper      mapper;

    @Override
    public List<ProductDTO> getAllProducts() {
        return mapper.toDTOList(repository.findAll());
    }

    @Override
    public ProductDTO getProductById(Long id) {
        Product product = repository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Product not found with id: " + id));
        return mapper.toDTO(product);
    }

    @Override
    public ProductDTO createProduct(CreateProductRequest
       request) {
        Product product = mapper.toEntity(request);
        Product saved   = repository.save(product);
        return mapper.toDTO(saved);
    }

    @Override
    public ProductDTO updateProduct(Long id,
       CreateProductRequest req) {
        Product product = repository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Product not found with id: " + id));
        mapper.updateEntity(req, product);
        return mapper.toDTO(repository.save(product));
    }

    @Override
    public void deleteProduct(Long id) {
        if (!repository.existsById(id))
            throw new RuntimeException("Product not found
               with id: " + id);
        repository.deleteById(id);
    }
}
```

Listing 2.6 – ProductServiceImpl.java — Logique métier avec Mapper

2.3.5 Controller adapté aux DTOs

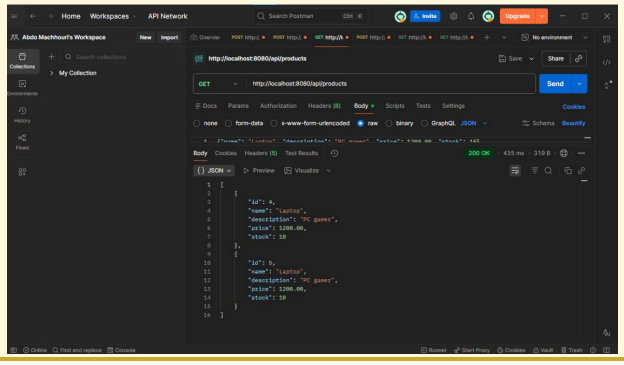
```java
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    @GetMapping
    public ResponseEntity<List<ProductDTO>> getAll() {
        return ResponseEntity.ok(productService.getAllProducts());
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }

    @PostMapping
    public ResponseEntity<ProductDTO> create(
            @Valid @RequestBody CreateProductRequest request) {
        ProductDTO created = productService.createProduct(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProductDTO> update(
            @PathVariable Long id,
            @Valid @RequestBody CreateProductRequest request) {
        return ResponseEntity.ok(
            productService.updateProduct(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

Listing 2.7 – ProductController.java — Avec DTOs

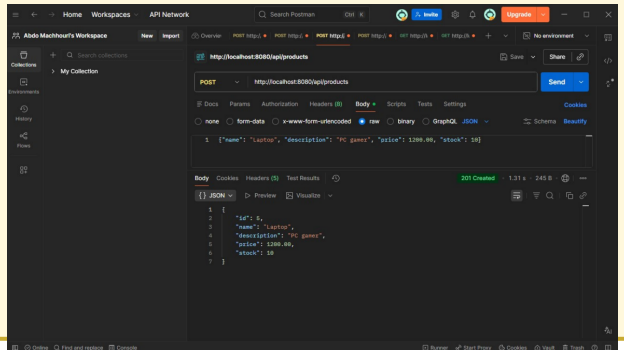
2.4 Tests avec Postman

2.4.1 Captures d'écran Postman

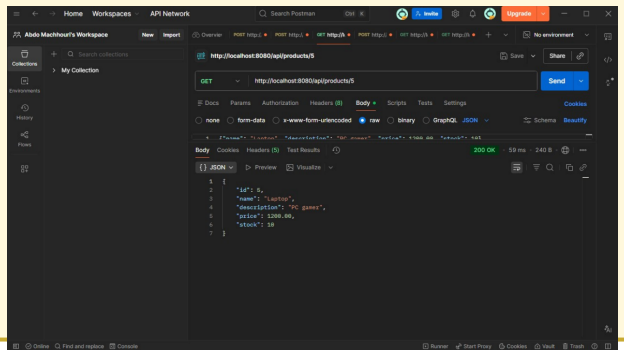
Capture Postman — GET /api/products — Réponse avec DTOs



Capture Postman — POST /api/products — Création via CreateProductRequest



Capture Postman — GET /api/customers/{id}/orders — Historique des commandes



2.5 Résumé du TP2

Résumé

Le TP2 marque une évolution qualitative importante de l'architecture. Les apports majeurs sont :

— **Pattern DTO** : découplage entre le modèle interne (Entity) et l'API exposée, permettant de choisir quels champs sont visibles.

— **MapStruct** : génération automatique du code de mapping à la compilation — performant et sans réflexion.

— **Interface de Service** : chaque module expose un contrat via une interface, facilitant les tests et le remplacement d'implémentation.

— **Boundaries modulaires** : l'Order Module consomme uniquement les services publics de Product et Customer, sans accès direct aux repositories — principe d'encapsulation inter-modules.

— **Préparation microservices** : cette structure modulaire rend la découpe en microservices naturelle et moins risquée.